

SECTION 3

INSTRUCTION SET SUMMARY

This section briefly describes the MC68030 instruction set. Refer to the MC68000PM/AD, *MC68000 Programmer's Reference Manual*, for complete details on the MC68030 instruction set.

3

The following paragraphs include descriptions of the instruction format and the operands used by instructions, followed by a summary of the instruction set. The integer condition codes and floating-point details are discussed. Programming examples for selected instructions are also presented.

3.1 INSTRUCTION FORMAT

All MC68030 instructions consist of at least one word; some have as many as 11 words (see Figure 3-1). The first word of the instruction, called the operation word, specifies the length of the instruction and the operation to be performed. The remaining words, called extension words, further specify the instruction and operands. These words may be floating-point command words, conditional predicates, immediate operands, extensions to the effective address mode specified in the operation word, branch displacements, bit number or bit field specifications, special register specifications, trap operands, pack/unpack constants, or argument counts.

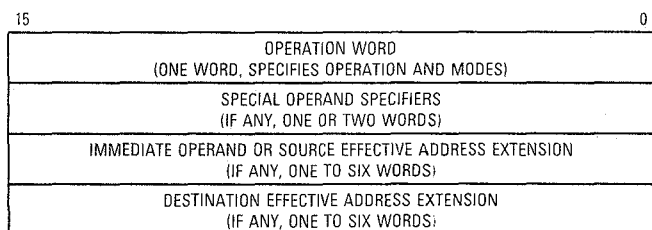


Figure 3-1. Instruction Word General Format

Besides the operation code, which specifies the function to be performed, an instruction defines the location of every operand for the function. Instructions specify an operand location in one of three ways:

1. **Register Specification** — A register field of the instruction contains the number of the register.
2. **Effective Address** — An effective address field of the instruction contains address mode information.
3. **Implicit Reference** — The definition of an instruction implies the use of specific registers.

The register field within an instruction specifies the register to be used. Other fields within the instruction specify whether the register selected is an address or data register and how the register is to be used. **SECTION 1 INTRODUCTION** contains register information.

Effective address information includes the registers, displacements, and absolute addresses for the effective address mode. **SECTION 2 DATA ORGANIZATION AND ADDRESSING CAPABILITIES** describes the effective address modes in detail.

Certain instructions operate on specific registers. These instructions imply the required registers.

3.2 INSTRUCTION SUMMARY

The instructions form a set of tools to perform the following operations:

Data Movement	Bit Field Manipulation
Integer Arithmetic	Binary-Coded Decimal Arithmetic
Logical	Program Control
Shift and Rotate	System Control
Bit Manipulation	Multiprocessor Communications

Each instruction type is described in detail in the following paragraphs.

The following notations are used in this section. In the operand syntax statements of the instruction definitions, the operand on the right is the destination operand.

An = any address register, A7–A0
 Dn = any data register, D7–D0
 Rn = any address or data register
 CCR = condition code register (lower byte of status register)
 cc = condition codes from CCR
 SR = status register
 SP = active stack pointer
 USP = user stack pointer
 ISP = supervisor/interrupt stack pointer
 MSP = supervisor/master stack pointer
 SSP = supervisor (master or interrupt) stack pointer
 DFC = destination function code register
 SFC = source function code register
 Rc = control register (VBR, SFC, DFC, CACR)
 MRc = MMU control register (SRP, URP, TC, DTT0, DTT1, ITT0, ITT1, MMUSR)
 MMUSR = MMU status register
 B, W, L = specifies a signed integer data type (twos complement) of byte, word, or long word
 S = single-precision real data format (32 bits)
 D = double-precision real data format (64 bits)
 X = extended-precision real data format (96 bits, 16 bits unused)
 P = packed BCD real data format (96 bits, 12 bytes)
 FPM, FPN = any floating-point data register, FP7–FP0
 PFcr = floating-point system control register (FPCR, FPSR, or FPIAR)
 k = a twos-complement signed integer (–64 to +17) that specifies the format of a number to be stored in the packed BCD format
 d = displacement; d16 is a 16-bit displacement
 <ea> = effective address
 list = list of registers, for example D3–D0
 #<data> = immediate data; a literal integer
 {offset:width} = bit field selection
 label = assemble program label
 [m] = bit m of an operand
 [m:n] = bits m through n of operand

X = extend (X) bit in CCR
 N = negative (N) bit in CCR
 Z = Zero (Z) bit in CCR
 V = overflow (V) bit in CCR
 C = carry (C) bit in CCR
 + = arithmetic addition or postincrement indicator
 − = arithmetic subtraction or predecrement indicator
 × = arithmetic multiplication
 ÷ = arithmetic division or conjunction symbol
 ~ = invert; operand is logically complemented
 ∧ = logical AND
 V = logical OR
 ⊕ = logical exclusive OR
 Dc = data register, D7–D0 used during compare
 Du = data register, D7–D0 used during update
 Dr, Dq = data registers, remainder or quotient of divide
 Dh, Dl = data registers, high- or low-order 32 bits of product
 MSW = most significant word
 LSW = least significant word
 MSB = most significant bit
 FC = function code
 {R/W} = read or write indicator
 [An] = address extensions

3.2.1 Data Movement Instructions

The MOVE instructions with their associated addressing modes are the basic means of transferring and storing addresses and data. MOVE instructions transfer byte, word, and long-word operands from memory to memory, memory to register, register to memory, and register to register. Address movement instructions (MOVE or MOVEA) transfer word and long-word operands and ensure that only valid address manipulations are executed. In addition to the general MOVE instructions, there are several special data movement instructions: move multiple registers (MOVEM), move peripheral data (MOVEP), move quick (MOVEQ), exchange registers (EXG), load effective address (LEA), push effective address (PEA), link stack (LINK), and unlink stack (UNLK).

Table 3-1 is a summary of the integer and floating-point data movement operations.

Table 3-1. Data Movement Operations

Instruction	Operand Syntax	Operand Size	Operation
EXG	Rn, Rn	32	Rn $\leftrightarrow$ Rn
LEA	<ea>, An	32	<ea> $\leftarrow$ An
LINK	An, #<d>	16, 32	SP $\leftarrow$ SP - 4 $\leftarrow$ SP; An $\leftarrow$ (SP); SP $\leftarrow$ An, SP + D $\leftarrow$ SP
MOVE MOVEA	<ea>, <ea> <ea>, An	8, 16, 32 16, 32 $\leftarrow$ 32	source $\leftarrow$ destination
MOVEM	list, <ea> <ea>, list	16, 32 16, 32 $\leftarrow$ 32	listed registers $\leftarrow$ destination source $\leftarrow$ listed registers
MOVEP	Dn, (d ₁₆ , An) (d ₁₆ , An), Dn	16, 32	Dn[31:24] $\leftarrow$ (An + d); Dn[23:16] $\leftarrow$ An + d + 2; Dn[15:8] $\leftarrow$ (An + d + 4); Dn[7:0] $\leftarrow$ (An + d + 6) (An - d) $\leftarrow$ Dn[31:24]; (An + d + 2) $\leftarrow$ Dn[23:16]; (An + d - 4) $\leftarrow$ Dn[15:8]; (An + d + 6) $\leftarrow$ Dn[7:0]
MOVEQ	#<data>, Dn	8 $\leftarrow$ 32	immediate data $\leftarrow$ destination
PEA	<ea>	32	SP $\leftarrow$ SP - 4 $\leftarrow$ SP; <ea> $\leftarrow$ (SP)
UNLK	An	32	An $\leftarrow$ SP; (SP) $\leftarrow$ An; SP $\leftarrow$ SP + 4 $\leftarrow$ SP

3.2.2 Integer Arithmetic Instructions

The integer arithmetic operations include the four basic operations of add (ADD), subtract (SUB), multiply (MUL), and divide (DIV) as well as arithmetic compare (CMP, CMPM, CMP2), clear (CLR), and negate (NEG). The instruction set includes ADD, CMP, and SUB instructions for both address and data operations with all operand sizes valid for data operations. Address operands consist of 16 or 32 bits. The clear and negate instructions apply to all sizes of data operands.

Signed and unsigned MUL and DIV instructions include:

- Word multiply to produce a long-word product
- Long-word multiply to produce a long-word or quad-word product
- Division of a long word divided by a word divisor (word quotient and word remainder)
- Division of a long word or quad word dividend by a long-word divisor (long-word quotient and long-word remainder)

A set of extended instructions provides multiprecision and mixed-size arithmetic. These instructions are add extended (ADDX), subtract extended (SUBX), sign extended (EXT), and negate binary with extend (NEGX). Refer to Table 3-2 for a summary of the integer arithmetic operations.

Table 3-2. Integer Arithmetic Operations

Instruction	Operand Syntax	Operand Size	Operation
ADD	Dn,(ea) (ea),Dn	8, 16, 32 8, 16, 32	source + destination $\nrightarrow$ destination
ADDA	(ea),An	16, 32	
ADDI	#{data),(ea)	8, 16, 32	immediate data + destination $\nrightarrow$ destination
ADDQ	#{data),(ea)	8, 16, 32	
ADDX	Dn,Dn -(An),-(An)	8, 16, 32 8, 16, 32	source + destination + X $\nrightarrow$ destination
CLR	(ea)	8, 16, 32	0 $\nrightarrow$ destination
CMP	(ea),Dn	8, 16, 32	destination - source
CMPA	(ea),An	16, 32	
CMPI	#{data),(ea)	8, 16, 32	destination - immediate data
CMPM	(An)+,(An)+	8, 16, 32	destination - source
CMP2	(ea),Rn	8, 16, 32	lower bound < = Rn < = upper bound
DIVS/DIVU	(ea),Dn (ea),Dr:Dq	32/16 $\nrightarrow$ 16:16 64/32 $\nrightarrow$ 32:32	destination/source $\nrightarrow$ destination (signed or unsigned)
DIVSL/DIVUL	(ea),Dq (ea),Dr:Dq	32/32 $\nrightarrow$ 32 32/32 $\nrightarrow$ 32:32	
EXT	Dn	8 $\nrightarrow$ 16 16 $\nrightarrow$ 32	sign extended destination $\nrightarrow$ destination
EXTB	Dn	8 $\nrightarrow$ 32	
MULS/MULU	(ea),Dn (ea),DI (ea),Dh:DI	16 $\times$ 16 $\nrightarrow$ 32 32 $\times$ 32 $\nrightarrow$ 32 32 $\times$ 32 $\nrightarrow$ 64	source $\times$ destination $\nrightarrow$ destination (signed or unsigned)
NEG	(ea)	8, 16, 32	0 - destination $\nrightarrow$ destination
NEGX	(ea)	8, 16, 32	0 - destination - X $\nrightarrow$ destination
SUB	(ea),Dn	8, 16, 32	destination = source $\nrightarrow$ destination
SUBA	Dn,(ea) (ea),An	8, 16, 32 16, 32	
SUBI	#{data),(ea)	8, 16, 32	destination - immediate data $\nrightarrow$ destination
SUBQ	#{data),(ea)	8, 16, 32	
SUBX	Dn,Dn -(An),-(An)	8, 16, 32 8, 16, 32	destination - source - X $\nrightarrow$ destination

3.2.3 Logical Instructions

The logical operation instructions (AND, OR, EOR, and NOT) perform logical operations with all sizes of integer data operands. A similar set of immediate instructions (ANDI, ORI, and EORI) provide these logical operations with all sizes of immediate data. The TST instruction compares the operand with zero arithmetically, placing the result in the condition code register. Table 3-3 summarizes the logical operations.

Table 3-3. Logical Operations

Instruction	Operand Syntax	Operand Size	Operation
AND	(ea),Dn Dn,(ea)	8, 16, 32 8, 16, 32	source $\wedge$ destination $\nrightarrow$ destination
ANDI	#<data>,<ea>	8, 16, 32	immediate data $\wedge$ destination $\nrightarrow$ destination
EOR	Dn,<data>,<ea>	8, 16, 32	source $\oplus$ destination $\nrightarrow$ destination
EORI	#(data),(ea)	8, 16, 32	immediate data $\oplus$ destination $\nrightarrow$ destination
NOT	(ea)	8, 16, 32	$\sim$ destination $\nrightarrow$ destination
OR	(ea),Dn Dn,(ea)	8, 16, 32 8, 16, 32	source $\vee$ destination $\nrightarrow$ destination
ORI	#(data),(ea)	8, 16, 32	immediate data $\vee$ destination $\nrightarrow$ destination
TST	(ea)	8, 16, 32	source — 0 to set condition codes

3.2.4 Shift and Rotate Instructions

The arithmetic shift instructions (ASR and ASL) and logical shift instructions (LSR and LSL) provide shift operations in both directions. The ROR, ROL, ROXR, and ROXL instructions perform rotate (circular shift) operations, with and without the extend bit. All shift and rotate operations can be performed on either registers or memory.

Register shift and rotate operations shift all operand sizes. The shift count may be specified in the instruction operation word (to shift from 1–8 places) or in a register (modulo 64 shift count).

Memory shift and rotate operations shift word-length operands one bit position only. The SWAP instruction exchanges the 16-bit halves of a register. Performance of shift/rotate instructions is enhanced so that use of the ROR and ROL instructions with a shift count of eight allows fast byte swapping. Table 3-4 is a summary of the shift and rotate operations.

Table 3-4. Shift and Rotate Operations

Instruction	Operand Syntax	Operand Size	Operation
ASL	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
ASR	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
LSL	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
LSR	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
ROL	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
ROR	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
ROXL	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
ROXR	Dn,Dn #(data),Dn (ea)	8, 16, 32 8, 16, 32 16	
SWAP	Dn	32	

3.2.5 Bit Manipulation Instructions

Bit manipulation operations are accomplished using the following instructions: bit test (BTST), bit test and set (BSET), bit test and clear (BCLR), and bit test and change (BCHG). All bit manipulation operations can be performed on either registers or memory. The bit number is specified as immediate data or in a data register. Register operands are 32 bits long, and memory operands are 8 bits long. In Table 3-5, the summary of the bit manipulation operations, Z refers to bit 2, the zero bit of the status register.

Table 3-5. Bit Manipulation Operations

Instruction	Operand Syntax	Operand Size	Operation
BCHG	Dn,⟨ea⟩ #⟨data⟩,⟨ea⟩	8, 32 8, 32	~ ((bit number) of destination) ♦ Z ♦ bit of destination
BCLR	Dn,⟨ea⟩ #⟨data⟩,⟨ea⟩	8, 32 8, 32	~ ((bit number) of destination) ♦ Z; 0 ♦ bit of destination
BSET	Dn,⟨ea⟩ #⟨data⟩,⟨ea⟩	8, 32 8, 32	~ ((bit number) of destination) ♦ Z; 1 ♦ bit of destination
BTST	Dn,⟨ea⟩ #⟨data⟩,⟨ea⟩	8, 32 8, 32	~ ((bit number) of destination) ♦ Z

3.2.6 Bit Field Instructions

The MC68030 supports variable-length bit field operations on fields of up to 32 bits. The bit field insert (BFINS) instruction inserts a value into a bit field. Bit field extract unsigned (BFEXTU) and bit field extract signed (BFEXTS) extract a value from the field. Bit field find first one (BFFFO) finds the first bit that is set in a bit field. Also included are instructions that are analogous to the bit manipulation operations; bit field test (BFTST), bit field test and set (BFSET), bit field test and clear (BFCLR), and bit field test and change (BFCHG). Table 3-6 is a summary of the bit field operations.

Table 3-6. Bit Field Operations

Instruction	Operand Syntax	Operand Size	Operation
BFCHG	⟨ea⟩ {offset:width}	1–32	~ Field ♦ Field
BFCLR	⟨ea⟩ {offset:width}	1–32	0's ♦ Field
BFEXTS	⟨ea⟩ {offset:width},Dn	1–32	Field ♦ Dn; Sign Extended
BFEXTU	⟨ea⟩ {offset:width},Dn	1–32	Field ♦ Dn; Zero Extended
BFFFO	⟨ea⟩ {offset:width},Dn	1–32	Scan for first bit set in field; offset ♦ Dn
BFINS	Dn,⟨ea⟩ {offset:width}	1–32	Dn ♦ Field
BFSET	⟨ea⟩ {offset:width}	1–32	1's ♦ Field
BFTST	⟨ea⟩ {offset:width}	1–32	Field MSB ♦ N; ~ (OR of all bits in field) ♦ Z

NOTE: All bit field instructions set the N and Z bits as shown for BFTST before performing the specified operation.

3.2.7 Binary-Coded Decimal Instructions

Five instructions support operations on binary-coded decimal (BCD) numbers. The arithmetic operations on packed BCD numbers are add decimal with extend (ABCD), subtract decimal with extend (SBCD), and negate decimal with extend (NBCD). PACK and UNPACK instructions aid in the conversion of byte encoded numeric data, such as ASCII or EBCDIC strings, to BCD data and vice versa. Table 3-7 is a summary of the BCD operations.

3

Table 3-7. BCD Operations

Instruction	Operand Syntax	Operand Size	Operation
ABCD	Dn,Dn	8	source ₁₀ + destination ₁₀ + X ∇ destination
	– (An), – (An)	8	
NBCD	(ea)	8	0 – destination ₁₀ – X ∇ destination
PACK	– (An), – (An)	16 ∇ 8	unpacked source + immediate data ∇ packed destination
	#(data) Dn,Dn,#(data)	16 ∇ 8	
SBCD	Dn,Dn	8	destination ₁₀ – source ₁₀ – X ∇ destination
	– (An), – (An)	8	
UNPK	– (An), – (An)	8 ∇ 16	packed source ∇ unpacked source unpacked source + immediate data ∇ unpacked destination
	#(data) Dn,Dn,#(data)	8 ∇ 16	

3.2.8 Program Control Instructions

A set of subroutine call and return instructions and conditional and unconditional branch instructions perform program control operations. The no operation instruction (NOP) may be used to force synchronization of the internal pipelines. Table 3-8 summarizes these instructions.

Table 3-8. Program Control Operations

Instruction	Operand Syntax	Operand Size	Operation
Integer and Floating-Point Conditional			
Bcc	<label>	8,16,32	if condition true, then PC + d ∇ PC
DBcc	Dn,<label>	16	if condition false, then Dn - 1 ∇ Dn if Dn $\neq$ -1, then PC + d ∇ PC
Scc	<ea>	8	if condition true, then 1's ∇ destination; else 0's ∇ destination
Unconditional			
BRA	<label>	8,16,32	PC + d ∇ PC
BSR	<label>	8,16,32	SP - 4 ∇ SP; PC ∇ (SP); PC + d ∇ PC
JMP	<ea>	none	destination ∇ PC
JSR	<ea>	none	SP - 4 ∇ SP; PC ∇ (SP); destination ∇ PC
NOP	none	none	PC + 2 ∇ PC
Returns			
RTD	#<d>	16	(SP) ∇ PC; SP + 4 + d ∇ SP
RTR	none	none	(SP) ∇ CCR; SP + 2 ∇ SP; (SP) ∇ PC; SP + 4 ∇ SP
RTS	none	none	(SP) ∇ PC; SP + 4 ∇ SP

Letters cc in the integer instruction mnemonics Bcc, DBcc, and Scc specify testing one of the following conditions:

CC — Carry clear	GE — Greater or equal
LS — Lower or same	PL — Plus
CS — Carry set	GT — Greater than
LT — Less than	T — Always true*
EQ — Equal	HI — Higher
MI — Minus	VC — Overflow clear
F — Never true*	LE — Less or equal
NE — Not equal	VS — Overflow set

*Not applicable to the Bcc or cpBcc instructions.

3.2.9 System Control Instructions

Privileged instructions, trapping instructions, and instructions that use or modify the condition code register (CCR) provide system control operations. Table 3-9 summarizes these instructions. The TRAPcc instruction uses the same conditional tests as the corresponding program control instructions. All of these instructions cause the processor to flush the instruction pipe.

3

Table 3-9. System Control Operations

Instruction	Operand Syntax	Operand Size	Operation
Privileged			
ANDI	#<data>,SR	16	immediate data $\wedge$ SR $\nrightarrow$ SR
EORI	#<data>,SR	16	immediate data $\oplus$ SR $\nrightarrow$ SR
MOVE	<ea>,SR SR,<ea>	16 16	source $\nrightarrow$ SR SR $\nrightarrow$ destination
MOVE	USP,An An,USP	32 32	USP $\nrightarrow$ An An $\nrightarrow$ USP
MOVEC	Rc,Rn Rn,Rc	32 32	Rc $\nrightarrow$ Rn Rn $\nrightarrow$ Rc
MOVES	Rn,<ea> <ea>,Rn	8,16,32	Rn $\nrightarrow$ destination using DFC source using SFC $\nrightarrow$ Rn
ORI	#<data>,SR	16	immediate data $\vee$ SR $\nrightarrow$ SR
RESET	none	none	assert RESET line
RTE	none	none	(SP) $\nrightarrow$ SR; SP + 2 $\nrightarrow$ SP; (SP) $\nrightarrow$ PC; SP + 4 $\nrightarrow$ SP; Restore stack according to format
STOP	#<data>	16	immediate data $\nrightarrow$ SR; STOP
Trap Generating			
BKPT	#<data>	none	run breakpoint cycle, then trap as illegal instruction
CHK	<ea>,Dn	16,32	if Dn < 0 or Dn > (ea), then CHK exception
CHK2	<ea>,Rn	8,16,32	if Rn < lower bound or Rn > upper bound, the CHK exception
ILLEGAL	none	none	SSP - 2 $\nrightarrow$ SSP; Vector Offset $\nrightarrow$ (SSP); SSP - 4 $\nrightarrow$ SSP; PC $\nrightarrow$ (SSP); SSP - 2 $\nrightarrow$ SSP; SR $\nrightarrow$ (SSP); Illegal Instruction Vector Address $\nrightarrow$ PC
TRAP	#<data>	none	SSP - 2 $\nrightarrow$ SSP; Format and Vector Offset $\nrightarrow$ (SSP) SSP - 4 $\nrightarrow$ SSP; PC $\nrightarrow$ (SSP); SSP - 2 $\nrightarrow$ SSP; SR $\nrightarrow$ (SSP); Vector Address $\nrightarrow$ PC
TRAPcc	none #<data>	none 16,32	if cc true, then TRAP exception
TRAPV	none	none	if V then take overflow TRAP exception
Condition Code Register			
ANDI	#<data>,CCR	8	immediate data $\wedge$ CCR $\nrightarrow$ CCR
EORI	#<data>,CCR	8	immediate data $\oplus$ CCR $\nrightarrow$ CCR
MOVE	<ea>,CCR CCR,<ea>	16 16	source $\nrightarrow$ CCR CCR $\nrightarrow$ destination
ORI	#<data>,CCR	8	immediate data $\vee$ CCR $\nrightarrow$ CCR

3.2.10 Memory Management Unit Instructions

The PFLUSH instructions flush the address translation caches (ATCs) and can optionally select only nonglobal entries for flushing. PTEST performs a search of the address translation tables, storing results in the MMU status register and loading the entry into the ATC. Table 3-10 summarizes these instructions.

Table 3-10. MMU Instructions

Instruction	Operand Syntax	Operand Size	Operation
PFLUSHA	none	none	Invalidate all ATC entries
PFLUSHA.N	none	none	Invalidate all nonglobal ATC entries
PFLUSH	(An)	none	Invalidate ATC entries at effective address
PFLUSH.N	(An)	none	Invalidate nonglobal ATC entries at effective address
PTEST	(An)	none	Information about logical address $\rightarrow$ MMU status register

3

3.2.11 Multiprocessor Instructions

The TAS, CAS, and CAS2 instructions coordinate the operations of processors in multiprocessing systems. These instructions use read-modify-write bus cycles to ensure uninterrupted updating of memory. Coprocessor instructions control the coprocessor operations. Table 3-11 lists these instructions.

Table 3-11. Multiprocessor Operations (Read-Modify-Write)

Instruction	Operand Syntax	Operand Size	Operation
Read-Modify-Write			
CAS	Dc,Du,<ea>	8,16,32	destination — Dc $\rightarrow$ CC; if Z then Du $\rightarrow$ destination else destination $\rightarrow$ Dc
CAS2	Dc1:Dc2, Du1:Du2, (Rn):(Rn)	8,16,32	dual operand CAS
TAS	<ea>	8	destination — 0; set condition codes; 1 $\rightarrow$ destination [7]
Coprocessor			
cpBcc	(label)	16, 32	if cpcc true then pc = d $\rightarrow$ PC
cpDBcc	(label),Dn	16	if cpcc false then Dn = 1 $\rightarrow$ Dn if Dn $\neq$ -1, then PC = d $\rightarrow$ PC
cpGEN	User Defined	User Defined	operand $\rightarrow$ coprocessor
cp RESTORE	(ea)	none	restore coprocessor state from (ea)
cpSAVE	(ea)	none	save coprocessor state at (ea)
cpScc	(ea)	8	if cpcc true, then 1's $\rightarrow$ destination; else 0's $\rightarrow$ destination
cpTRAPcc	none #(data)	none 16, 32	if cpcc true then TRAPcc exception

3.3 INTEGER CONDITION CODES

The CCR portion of the SR contains five bits which indicate the results of many integer instructions. Program and system control instructions use certain combinations of these bits to control program and system flow.

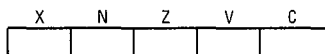
The first four bits represent a condition resulting from a processor operation. The X bit is an operand for multiprecision computations; when it is used, it is set to the value of the C bit. The carry bit and the multiprecision extend bit are separate in the M68000 Family to simplify programming techniques that use them (refer to Table 3-8 as an example).

The condition codes were developed to meet two criteria:

- Consistency — across instructions, uses, and instances
- Meaningful Results — no change unless it provides useful information

Consistency across instructions means that all instructions that are special cases of more general instructions affect the condition codes in the same way. Consistency across instances means that all instances of an instruction affect the condition codes in the same way. Consistency across uses means that conditional instructions test the condition codes similarly and provide the same results, regardless of whether the condition codes are set by a compare, test, or move instruction.

In the instruction set definitions, the CCR is shown as follows:



where:

X (extend)

Set to the value of the C bit for arithmetic operations. Otherwise not affected or set to a specified result.

N (negative)

Set if the most significant bit of the result is set. Cleared otherwise.

Z (zero)

Set if the result equals zero. Cleared otherwise.

V (overflow)

Set if arithmetic overflow occurs. This implies that the result cannot be represented in the operand size. Cleared otherwise.

C (carry)

Set if a carry out of the most significant bit of the operand occurs for an addition. Also set if a borrow occurs in a subtraction. Cleared otherwise.

3.3.1 Condition Code Computation

Most operations take a source operand and a destination operand, compute, and store the result in the destination location. Single-operand operations take a destination operand, compute, and store the result in the destination location. Table 3-12 lists each instruction and how it affects the condition code bits.

Table 3-12. Condition Code Computations (Sheet 1 of 2)

Operations	X	N	Z	V	C	Special Definition
ABCD	*	U	?	U	?	C = Decimal Carry $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
ADD, ADDI, ADDQ	*	*	*	?	?	$V = Sm \wedge Dm \wedge \overline{Rm} \vee \overline{Sm} \wedge Dm \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$
ADDX	*	*	?	?	?	$V = Sm \wedge Dm \wedge \overline{Rm} \vee \overline{Sm} \wedge Dm \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
AND, ANDI, EOR, EORI, MOVEQ, MOVE, OR, ORI, CLR, EXT, NOT, TAS, TST	—	*	*	0	0	
CHK	—	*	U	U	U	
CHK2, CMP2	—	U	?	U	?	$Z = (R = LB) \vee (R = UB)$ $C = (LB < = UB) \wedge ((R < LB) \vee (R > UB))$ $\vee (UB < LB) \wedge (R > UB) \wedge (R < LB)$
SUB, SUBI, SUBQ	*	*	*	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge Dm \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$
SUBX	*	*	?	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge Dm \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
CAS, CAS2, CMP, CMPI, CMPM	—	*	*	?	?	$V = \overline{Sm} \wedge Dm \wedge \overline{Rm} \vee Sm \wedge Dm \wedge Rm$ $C = Sm \wedge Dm \vee \overline{Rm} \wedge Dm \vee Sm \wedge \overline{Rm}$
DIVS, DUVI	—	*	*	?	0	V = Division Overflow
MULS, MULU	—	*	*	?	0	V = Multiplication Overflow
SBCD, NBCD	*	U	?	U	?	C = Decimal Borrow $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$
NEG	*	*	*	?	?	$V = Dm \wedge Rm$ $C = Dm \vee Rm$
NEGX	*	*	?	?	?	$V = Dm \wedge Rm$ $C = Dm \vee Rm$ $Z = Z \wedge \overline{Rm} \wedge \dots \wedge \overline{R0}$

Table 3-12. Condition Code Computations (Sheet 2 of 2)

Operations	X	N	Z	V	C	Special Definition
BTST, BCHG, BSET, BCLR	—	—	?	—	—	$Z = \overline{D_n}$
BFTST, BFCHG, BFSET, BFCLR	—	?	?	0	0	$N = D_m$ $Z = D_m \wedge \overline{D_{m-1}} \wedge \dots \wedge \overline{D_0}$
BFEXTS, BFEXTU, BFFFO	—	?	?	0	0	$N = S_m$ $Z = S_m \wedge \overline{S_{m-1}} \wedge \dots \wedge \overline{S_0}$
BFINS	—	?	?	0	0	$N = D_m$ $Z = D_m \wedge \overline{D_{m-1}} \wedge \dots \wedge \overline{D_0}$
ASL	*	*	*	?	?	$V = D_m \wedge (\overline{D_{m-1}} \vee \dots \vee \overline{D_{m-r}}) \vee \overline{D_m} \wedge$ $(D_{m-1} \vee \dots \vee D_{m-r})$ $C = D_{m-r+1}$
ASL (R=0)	—	*	*	0	0	
LSL, ROXL	*	*	*	0	?	$C = D_{m-r+1}$
LSR (r=0)	—	*	*	0	0	
ROXL (r=0)	—	*	*	0	?	$C = X$
ROL	—	*	*	0	?	$C = D_{m-r+1}$
ROL (r=0)	—	*	*	0	0	
ASR, LSR, ROXR	*	*	*	0	?	$C = D_r - 1$
ASR, LSR (r=0)	—	*	*	0	0	
ROXR (r=0)	—	*	*	0	?	$C = X$
ROR	—	*	*	0	?	$C = D_r - 1$
ROR (r=0)	—	*	*	0	0	

— = Not Affected

U = Undefined, Result Meaningless

? = Other — See Special Definition

* = General Case

X = C

N = R_m

Z = $\overline{R_m} \wedge \dots \wedge \overline{R_0}$

S_m = Source Operand — Most Significant Bit

D_m = Destination Operand — Most Significant Bit

R_m = Result Operand — Most Significant Bit

R = Register Tested

n = Bit Number

r = Shift Count

LB = Lower Bound

UB = Upper Bound

$\wedge$ = Boolean AND

$\vee$ = Boolean OR

$\overline{R_m}$ = NOT R_m

3.3.2 Conditional Tests

Table 3-13 lists the condition names, encodings, and tests for the conditional branch and set instructions. The test associated with each condition is a logical formula using the current states of the condition codes. If this formula evaluates to one, the condition is true. If the formula evaluates to zero, the condition is false. For example, the T condition is always true, and the EQ condition is true only if the Z bit condition code is currently true.

Table 3-13. Conditional Tests

Mnemonic	Condition	Encoding	Test
T*	True	0000	1
F*	False	0001	0
HI	High	0010	$\overline{C} \cdot \overline{Z}$
LS	Low or Same	0011	$C + Z$
CC(HS)	Carry Clear	0100	$\overline{C}$
CS(LO)	Carry Set	0101	C
NE	Not Equal	0110	$\overline{Z}$
EQ	Equal	0111	Z
VC	Overflow Clear	1000	$\overline{V}$
VS	Overflow Set	1001	V
PL	Plus	1010	$\overline{N}$
MI	Minus	1011	N
GE	Greater or Equal	1100	$N \cdot V + \overline{N} \cdot \overline{V}$
LT	Less Than	1101	$N \cdot \overline{V} + \overline{N} \cdot V$
GT	Greater Than	1110	$N \cdot V \cdot \overline{Z} + \overline{N} \cdot \overline{V} \cdot \overline{Z}$
LE	Less or Equal	1111	$Z + N \cdot \overline{V} + \overline{N} \cdot V$

• = Boolean AND

+ = Boolean OR

$\overline{N}$ = Boolean NOT N

*Not available for the Bcc instruction.

3.4 INSTRUCTION SET SUMMARY

Table 3-14 provides a alphabetized listing of the MC68030 instruction set listed by opcode, operation, and syntax.

Table 3-14 use notational conventions for the operands, the subfields and qualifiers, and the operations performed by the instructions. In the syntax descriptions, the left operand is the source operand, and the right operand is the destination operand. The following list contains the notations used in Table 3-14.

Notation for operands:

- PC—Program counter
- SR—Status register
- V—Overflow condition code
- Immediate Data—Immediate data from the instruction
- Source—Source contents
- Destination—Destination contents
- Vector—Location of exception vector
- + inf—Positive infinity
- inf—Negative infinity
- <fmt>—Operand data format: byte (B), word (W), long (L), single (S), double (D), extended (X), or packed (P).
- FPm—One of eight floating-point data registers (always specifies the source register)
- FPn—One of eight floating-point data registers (always specifies the destination register)

Notation for subfields and qualifiers:

- <bit> of <operand>—Selects a single bit of the operand
- <ea>{offset:width}—Selects a bit field
- (<operand>)—The contents of the referenced location
- <operand>₁₀—The operand is binary coded decimal, operations are performed in decimal
- (<address register>)—The register indirect operator
- (<address register>)—Indicates that the operand register points to the memory
- (<address register>)+—Location of the instruction operand — the optional mode qualifiers are –, +, (d), and (d,ix)
- #xxx or #<data>—Immediate data that follows the instruction word(s)

Notations for operations that have two operands, written $\langle \text{operand} \rangle \langle \text{op} \rangle \langle \text{operand} \rangle$, where $\langle \text{op} \rangle$ is one of the following:

- ◆—The source operand is moved to the destination operand
- ◄—The two operands are exchanged
- +—The operands are added
- The destination operand is subtracted from the source operand
- ×—The operands are multiplied
- ÷—The source operand is divided by the destination operand
- <—Relational test, true if source operand is less than destination operand
- >—Relational test, true if source operand is greater than destination operand
- V—Logical OR
- ⊕—Logical exclusive OR
- Λ—Logical AND
- shifted by, rotated by—The source operand is shifted or rotated by the number of positions specified by the second operand

Notation for single-operand operations:

- ~ $\langle \text{operand} \rangle$ —The operand is logically complemented
- $\langle \text{operand} \rangle$ sign-extended—The operand is sign extended; all bits of the upper portion are made equal to the high-order bit of the lower portion
- $\langle \text{operand} \rangle$ tested—The operand is compared to zero, and the condition codes are set appropriately

Notation for other operations:

- TRAP—Equivalent to Format/Offset Word ◆ (SSP); SSP – 2 ◆ SSP; PC ◆ (SSP); SSP – 4 ◆ SSP; SR ◆ (SSP); SSP – 2 ◆ SSP; (vector) ◆ PC
- STOP—Enter the stopped state, waiting for interrupts
- If $\langle \text{condition} \rangle$ then—The condition is tested. If true, the operations $\langle \text{operations} \rangle$ after “then” are performed. If the condition is false and the optional “else” clause is present, the operations after “else” are performed. If the condition is false and else is omitted, the instruction performs no operation. Refer to the Bcc instruction description as an example.

Table 3-14. Instruction Set Summary (Sheet 1 of 5)

Opcode	Operation	Syntax
ABCD	Source ₁₀ + Destination ₁₀ + X ∇ Destination	ABCD Dy,Dx ABCD – (Ay), – (Ax)
ADD	Source + Destination ∇ Destination	ADD (ea),Dn ADD Dn,(ea)
ADDA	Source + Destination ∇ Destination	ADDA (ea),An
ADDI	Immediate Data + Destination ∇ Destination	ADDI #(data),(ea)
ADDQ	Immediate Data + Destination ∇ Destination	ADDQ #(data),(ea)
ADDX	Source + Destination + X ∇ Destination	ADDX Dy,Dx ADDX – (Ay), – (Ax)
AND	Source $\wedge$ Destination ∇ Destination	AND (ea),Dn AND Dn,(ea)
ANDI	Immediate Data $\wedge$ Destination ∇ Destination	ANDI #(data),(ea)
ANDI to CCR	Source $\wedge$ CCR ∇ CCR	ANDI #(data),CCR
ANDI to SR	If supervisor state the Source $\wedge$ SR ∇ SR else TRAP	ANDI #(data),SR
ASL,ASR	Destination Shifted by (count) ∇ Destination	ASd Dx,Dy ASd #(data),Dy ASd (ea)
Bcc	If (condition true) then PC + d ∇ PC	Bcc (label)
BCHG	$\sim$ ((number) of Destination) ∇ Z; $\sim$ ((number) of Destination) ∇ (bit number) of Destination	BCHG Dn,(ea) BCHG #(data),(ea)
BCLR	$\sim$ ((bit number) of Destination) ∇ Z; 0 ∇ (bit number) of Destination	BCLR Dn,(ea) BCLR #(data),(ea)
BFCHG	$\sim$ ((bit field) of Destination) ∇ (bit field) of Destination	BFCHG (ea){offset:width}
BFCLR	0 ∇ (bit field) of Destination	BFCLR (ea){offset:width}
BFEXTS	(bit field) of Source ∇ Dn	BFEXTS (ea){offset:width},Dn
BFEXTU	(bit offset) of Source ∇ Dn	BFEXTU (ea){offset:width},Dn
BFFFO	(bit offset) of Source Bit Scan ∇ Dn	BFFFO (ea){offset:width},Dn
BFINS	Dn ∇ (bit field) of Destination	BFINS Dn,(ea){offset:width}
BFSET	1s ∇ (bit field) of Destination	BFSET (ea){offset:width}
BFTST	(bit field) of Destination	BFTST (ea){offset:width}
BKPT	Run breakpoint acknowledge cycle; TRAP as illegal instruction	BKPT #(data)
BRA	PC + d ∇ PC	BRA (label)
BSET	$\sim$ ((bit number) of Destination) ∇ Z; 1 ∇ (bit number) of Destination	BSET Dn,(ea) BSET #(data),(ea)
BSR	SP – 4 ∇ SP; PC ∇ (SP); PC + d ∇ PC	BSR (label)
BTST	$\sim$ ((bit number) of Destination) ∇ Z;	BTST Dn,(ea) BTST #(data),(ea)

Table 3-14. Instruction Set Summary (Sheet 2 of 5)

Opcode	Operation	Syntax
CAS CAS2	CAS Destination — Compare Operand ∇ cc; if Z, Update Operand ∇ Destination else Destination ∇ Compare Operand CAS2 Destination 1 — Compare 1 ∇ cc; if Z, Destination 2 — Compare ∇ cc; if Z, Update 1 ∇ Destination 1; Update 2 ∇ Destination 2 else Destination 1 ∇ Compare 1; Destination 2 ∇ Compare 2	CAS Dc,Du,(ea) CAS2 Dc1:Dc2,Du1:Du2,(Rn1):(Rn2)
CHK	If Dn < 0 or Dn > Source then TRAP	CHK (ea),Dn
CHK2	If Rn < lower bound or Rn > upper bound then TRAP	CHK2 (ea),Rn
CLR	0 ∇ Destination	CLR (ea)
CMP	Destination — Source ∇ cc	CMP (ea),Dn
CMPA	Destination — Source	CMPA (ea),An
CMPI	Destination — Immediate Data	CMPI #(data),(ea)
CMPM	Destination — Source ∇ cc	CMPM (Ay) + ,(Ax) +
CMP2	Compare Rn < lower-bound or Rn > upper-bound and Set Condition Codes	CMP2 (ea),Rn
cpBcc	If cpcc true then scanPC + d ∇ PC	cpBcc (label)
cpDBcc	If cpcc false then (Dn - 1 ∇ Dn; If Dn $\neq$ - 1 then scanPC + d ∇ PC)	cpDBcc Dn,(label)
cpGEN	Pass Command Word to Coprocessor	cpGEN (parameters as defined by co-processor)
cpRESTORE	If supervisor state then Restore Internal State of Coprocessor else TRAP	cpRESTORE (ea)
cpSAVE	If supervisor state then Save Internal State of Coprocessor else TRAP	cpSAVE (ea)
cpScc	If cpcc true then 1s ∇ Destination else 0s ∇ Destination	cpScc (ea)
cpTRAPcc	If cpcc true then TRAP	cpTRAPcc cpTRAPcc #(data)
DBcc	If condition false then (Dn - 1 ∇ Dn; If Dn $\neq$ - 1 then PC + d ∇ PC)	DBcc Dn,(label)
DIVS DIVSL	Destination/Source ∇ Destination	DIVS.W (ea),Dn 32/16 ∇ 16r:16q DIVS.L (ea),Dq 32/32 ∇ 32q DIVS.L (ea),Dr:Dq 64/32 ∇ 32r:32q DIVSL.L (ea),Dr:Dq 32/32 ∇ 32r:32q
DIVU DIVUL	Destination/Source ∇ Destination	DIVU.W (ea),Dn 32/16 ∇ 16r:16q DIVU.L (ea),Dq 32/32 ∇ 32q DIVU.L (ea),Dr:Dq 64/32 ∇ 32r:32q DIVUL.L (ea),Dr:Dq 32/32 ∇ 32r:32q
EOR	Source $\oplus$ Destination ∇ Destination	EOR Dn,(ea)
EORI	Immediate Data $\oplus$ Destination ∇ Destination	EORI #(data),(ea)

Table 3-14. Instruction Set Summary (Sheet 3 of 5)

Opcode	Operation	Syntax
EORI to CCR	Source $\oplus$ CCR $\rightarrow$ CCR	EORI #(data),CCR
EORI to SR	If supervisor state the Source $\oplus$ SR $\rightarrow$ SR else TRAP	EORI #(data),SR
EXG	Rx $\leftrightarrow$ Ry	EXG Dx,Dy EXG Ax,Ay EXG Dx,Ay EXG Ay,Dx
EXT EXTB	Destination Sign-Extended $\rightarrow$ Destination	EXT.W Dn extend byte to word EXT.L L Dn extend word to long word EXTB.L Dn extend byte to long word
ILLEGAL	SSP - 2 $\rightarrow$ SSP; Vector Offset $\rightarrow$ (SSP); SSP - 4 $\rightarrow$ SSP; PC $\rightarrow$ (SSP); SSP - 2 $\rightarrow$ SSP; SR $\rightarrow$ (SSP); Illegal Instruction Vector Address $\rightarrow$ PC	ILLEGAL
JMP	Destination Address $\rightarrow$ PC	JMP (ea)
JSR	SP - 4 $\rightarrow$ SP; PC $\rightarrow$ (SP) Destination Address $\rightarrow$ PC	JSR (ea)
LEA	(ea) $\rightarrow$ An	LEA (ea),An
LINK	SP - 4 $\rightarrow$ SP; An $\rightarrow$ (SP) SP $\rightarrow$ An, SP + d $\rightarrow$ SP	LINK An,#(displacement)
LSL,LSR	Destination Shifted by (count) $\rightarrow$ Destination	LSd ⁵ Dx,Dy LSd ⁵ #(data),Dy LSd ⁵ (ea)
MOVE	Source $\rightarrow$ Destination	MOVE (ea),(ea)
MOVEA	Source $\rightarrow$ Destination	MOVEA (ea),An
MOVE from CCR	CCR $\rightarrow$ Destination	MOVE CCR,(ea)
MOVE to CCR	Source $\rightarrow$ CCR	MOVE (ea),CCR
MOVE from SR	If supervisor state then SR $\rightarrow$ Destination else TRAP	MOVE SR,(ea)
MOVE to SR	If supervisor state then Source $\rightarrow$ SR else TRAP	MOVE (ea),SR
MOVE USP	If supervisor state then USP $\rightarrow$ An or An $\rightarrow$ USP else TRAP	MOVE USP,An MOVE An,USP
MOVEC	If supervisor state then Rc $\rightarrow$ Rn or Rn $\rightarrow$ Rc else TRAP	MOVEC Rc,Rn MOVEC Rn,Rc
MOVEM	Registers $\rightarrow$ Destination Source $\rightarrow$ Registers	MOVEM register list,(ea) MOVEM (ea),register list
MOVEP	Source $\rightarrow$ Destination	MOVEP Dx,(d,Ay) MOVEP (d,Ay),Dx
MOVEQ	Immediate Data $\rightarrow$ Destination	MOVEQ #(data),Dn

Table 3-14. Instruction Set Summary (Sheet 4 of 5)

Opcode	Operation	Syntax
MOVES	If supervisor state then Rn $\blacktriangleright$ Destination [DFC] or Source [SFC] $\blacktriangleright$ Rn else TRAP	MOVES Rn,(ea) MOVES (ea),Rn
MULS	Source $\times$ Destination $\blacktriangleright$ Destination	MULS.W (ea),Dn 16 $\times$ 16 $\blacktriangleright$ 32 MULS.L (ea),DI 32 $\times$ 32 $\blacktriangleright$ 32 MULS.L (ea),Dh:DI 32 $\times$ 32 $\blacktriangleright$ 64
MULU	Source $\times$ Destination $\blacktriangleright$ Destination	MULU.W (ea),Dn 16 $\times$ 16 $\blacktriangleright$ 32 MULU.L (ea),DI 32 $\times$ 32 $\blacktriangleright$ 32 MULU.L (ea),Dh:DI 32 $\times$ 32 $\blacktriangleright$ 64
NBCD	0 – (Destination ₁₀) – X $\blacktriangleright$ Destination	NBCD (ea)
NEG	0 – (Destination) $\blacktriangleright$ Destination	NEG (ea)
NEGX	0 – (Destination) – X $\blacktriangleright$ Destination	NEGX (ea)
NOP	None	NOP
NOT	$\sim$ Destination $\blacktriangleright$ Destination	NOT (ea)
OR	Source V Destination $\blacktriangleright$ Destination	OR (ea),Dn OR Dn,(ea)
ORI	Immediate Data V Destination $\blacktriangleright$ Destination	ORI #(data),(ea)
ORI to CCR	Source V CCR $\blacktriangleright$ CCR	ORI #(data),CCR
ORI to SR	If supervisor state then Source V SR $\blacktriangleright$ SR else TRAP	ORI #(data),SR
PACK	Source (Unpacked BCD) + adjustment $\blacktriangleright$ Destination (Packed BCD)	PACK – (Ax), – (Ay), #(adjustment) PACK Dx,Dy, #(adjustment)
PEA	Sp – 4 $\blacktriangleright$ SP; (ea) $\blacktriangleright$ (SP)	PEA (ea)
PFLUSH	If supervisor state then invalidate instruction and data ATC entries for destination address else TRAP	PFLUSH (An) PFLUSHN (An) PFLUSHA PFLUSHAN
PLOAD	If supervisor state then entry $\blacktriangleright$ ATC else TRAP	PLOADR (function code),(ea) PLOADW (function code),(ea)
PMOVE	If supervisor state then (Source) $\blacktriangleright$ MRn or MRn $\blacktriangleright$ (Destination)	PMOVE MRn,(ea) PMOVE (ea),MRn PMOVEFD (ea),MRn
PTEST	If supervisor state then logical address status $\blacktriangleright$ MMUSR; entry $\blacktriangleright$ ATC else TRAP	PTESTR (An) PTESTW (An)
RESET	If supervisor state then Assert $\overline{\text{RSTO}}$ Line else TRAP	RESET
ROL,ROR	Destination Rotated by (count) $\blacktriangleright$ Destination	ROd ⁵ Rx,Dy ROd ⁵ #(data),Dy ROd ⁵ (ea)
ROXL,ROXR	Destination Rotated with X by (count) $\blacktriangleright$ Destination	ROXd ⁵ Dx,Dy ROXd ⁵ #(data),Dy ROXd ⁵ (ea)

Table 3-14. Instruction Set Summary (Sheet 5 of 5)

Opcode	Operation	Syntax
RTD	(SP) ∇ PC; SP + 4 ∇ SP	RTD #(displacement)
RTE	If supervisor state the (SP) ∇ SR; SP + 2 ∇ SP; (SP) ∇ PC; SP + 4 ∇ SP; restore state and deallocate stack according to (SP) else TRAP	RTE
RTR	(SP) ∇ CCR; SP + 2 ∇ SP; (SP) ∇ PC; SP + 4 ∇ SP	RTR
RTS	(SP) ∇ PC; SP + 4 ∇ SP	RTS
SBCD	Destination ₁₀ – Source ₁₀ – X ∇ Destination	SBCD Dx,Dy SBCD – (Ax), – (Ay)
Scc	If Condition True then 1s ∇ Destination else 0s ∇ Destination	Scc (ea)
STOP	If supervisor state then Immediate Data ∇ SR; STOP else TRAP	STOP #(data)
SUB	Destination – Source ∇ Destination	SUB (ea),Dn SUB Dn,(ea)
SUBA	Destination – Source ∇ Destination	SUBA (ea),An
SUBI	Destination – Immediate Data ∇ Destination	SUBI #(data),(ea)
SUBQ	Destination – Immediate Data ∇ Destination	SUBQ #(data),(ea)
SUBX	Destination – Source – X ∇ Destination	SUBX Dx,Dy SUBX – (Ax), – (Ay)
SWAP	Register [31:16] ∇ Register [15:0]	SWAP Dn
TAS	Destination Tested ∇ Condition Codes; 1 ∇ bit 7 of Destination	TAS (ea)
TRAP	SSP – 2 ∇ SSP; Format/Offset ∇ (SSP); SSP – 4 ∇ SSP; PC ∇ (SSP); SSP – 2 ∇ SSP; SR ∇ (SSP); Vector Address ∇ PC	TRAP #(vector)
TRAPcc	If cc then TRAP	TRAPcc TRAPcc.W #(data) TRAPcc.L #(data)
TRAPV	If V then TRAP	TRAPV
TST	Destination Tested ∇ Condition Codes	TST (ea)
UNLK	An ∇ SP; (SP) ∇ An; SP + 4 ∇ SP	UNLK An
UNPK	Source (Packed BCD) + adjustment ∇ Destination (Unpacked BCD)	UNPACK – (Ax), – (Ay), #(adjustment) UNPACK Dx,Dy, #(adjustment)

NOTES:

1. Specifies either the instruction (IC), data (DC), or IC/DC caches.
2. Where r is rounding precision, S or D.
3. A list of any combination of the eight floating-point data registers, with individual register names separated by a slash (/); and/or contiguous blocks of registers specified by the first and last register names separated by a dash (–).
4. A list of any combination of the three floating-point system control registers (FPCR, FPSR, and FPIAR) with individual register names separated by a slash (/).
5. where d is direction, L or R.



3.5 INSTRUCTION EXAMPLES

The following paragraphs provide examples of how to use selected instructions.

3.5.1 Using the CAS and CAS2 Instructions

The CAS instruction compares the value in a memory location with the value in a data register, and copies a second data register into the memory location if the compared values are equal. This provides a means of updating system counters, history information, and globally shared pointers. The instruction uses an indivisible read-modify-write cycle; after CAS reads the memory location, no other instruction can change that location before CAS has written the new value. This provides security in single-processor systems, in multitasking environments, and in multiprocessor environments. In a single-processor system, the operation is protected from instructions of an interrupt routine. In a multitasking environment, no other task can interfere with writing the new value of a system variable. In a multiprocessor environment, the other processors must wait until the CAS instruction completes before accessing a global pointer.

The following code fragment shows a routine to maintain a count, in location SYS_CNTR, of the executions of an operation that may be performed by any process or processor in a system. The routine obtains the current value of the count in register D0 and stores the new count value in register D1. The CAS instruction copies the new count into SYS_CNTR if it is valid. However, if another user has incremented the counter between the time the count was stored and the read-modify-write cycle of the CAS instruction, the write portion of the cycle copies the new count in SYS_CNTR into D0, and the routine branches to repeat the test. The following code sequence guarantees that SYS_CNTR is correctly incremented.

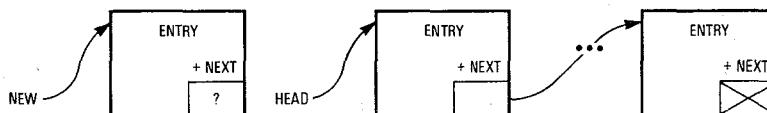
	MOVE.W	SYS_CNTR,D0	get the old value of the counter
INC_LOOP	MOVE.W	D0,D1	make a copy of it
	ADDQ.W	#1,D1	and increment it
	CAS.W	D0,D1,SYS_CNTR	if counter value is still the same, update it
	BNE	INC_LOOP	if not, try again

The CAS and CAS2 instructions together allow safe operations in the manipulation of system linked lists. Controlling a single location, HEAD in the example, manages a last-in-first-out linked list (see Figure 3-2). If the list is empty, HEAD contains the NULL pointer (0); otherwise, HEAD contains the address of the element most recently added to the list. The code fragment shown in Figure 3-2 illustrates the code for inserting an element. The MOVE instructions load the address in location HEAD into D0 and into the NEXT pointer in the element being inserted, and the address of the new element into D1. The CAS instruction stores the address of the inserted element into location HEAD if the address in HEAD remains unaltered. If HEAD contains a new address, the instruction loads the new address into D0 and branches to the second MOVE instruction to try again.

The CAS2 instruction is similar to the CAS instruction except that it performs two comparisons and updates two variables when the results of the comparisons are equal. If the results of both comparisons are equal, CAS2 copies new values into the destination addresses. If the result of either comparison is not equal, the instruction copies the values in the destination addresses into the compare operands.

SINSERT			ALLOCATE NEW ENTRY, ADDRESS IN A1
	MOVE.L	HEAD,D0	MOVE HEAD POINTER VALUE TO D0
SILOOP	MOVE.L	D0,(NEXT,A1)	ESTABLISH FORWARD LINK IN NEW ENTRY
	MOVE.L	A1,D1	MOVE NEW ENTRY POINTER VALUE TO D1
	CAS.L	D0,D1,HEAD	IF WE STILL POINT TO TOP OF STACK, UPDATE THE HEAD POINTER.
	BNE	SILOOP	IF NOT, TRY AGAIN

BEFORE INSERTING AN ELEMENT:



AFTER INSERTING AN ELEMENT:

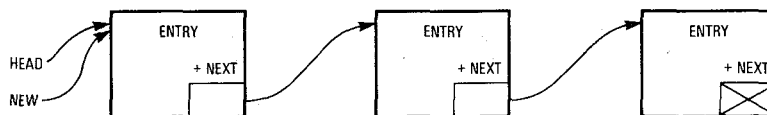


Figure 3-2. Linked List Insertion

The next code (see Figure 3-3) fragment shows the use of a CAS2 instruction to delete an element from a linked list. The first LEA instruction loads the effective address of HEAD into A0. The MOVE instruction loads the address in pointer HEAD into D0. The TST instruction checks for an empty list, and the BEQ instruction branches to a routine at label SDEEMPTY if the list is empty. Otherwise, a second LEA instruction loads the address of the NEXT pointer in the newest element on the list into A1, and the following MOVE instruction loads the pointer contents into D1. The CAS2 instruction compares the address of the newest structure to the value in HEAD and the address in D1 to the pointer in the address in A1. If no element has been inserted or deleted by another routine while this routine has been executing, the results of these comparisons are equal, and the CAS2 instruction stores the new value into location HEAD. If an element has been inserted or deleted, the CAS2 instruction loads the new address in location HEAD into D0, and the BNE instruction branches to the TST instruction to try again.

SDELETE	LEA	HEAD,A0	LOAD ADDRESS OF HEAD POINTER INTO A0
	MOVE.L	(A0),D0	MOVE VALUE OF HEAD POINTER INTO D0
SDLOOP	TST.L	D0	CHECK FOR NULL HEAD POINTER
	BEQ	SDEEMPTY	IF EMPTY, NOTHING TO DELETE
	LEA	(NEXT,D0),A1	LOAD ADDRESS OF FORWARD LINK INTO A1
	MOVE.L	(A1),D1	PUT FORWARD LINK VALUE IN D1
	CAS2.L	D0:D1,D1:D1,(A0):(A1)	IF STILL POINT TO ENTRY TO BE DELETED, THEN UPDATE HEAD AND FORWARD POINTERS
	BNE	SDLOOP	IF NOT, TRY AGAIN
SDEEMPTY			SUCCESSFUL DELETION, ADDRESS OF DELETED ENTRY IN D0 (MAY BE NULL)

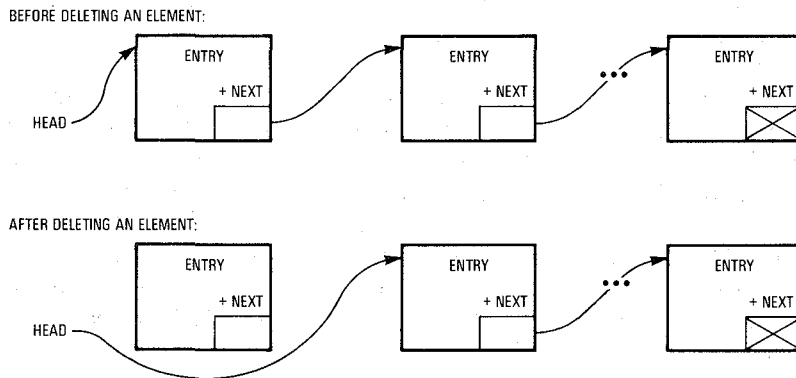


Figure 3-3. Linked List Deletion

The CAS2 instruction can also be used to correctly maintain a first-in-first-out doubly linked list. A doubly linked list needs two controlled locations, LIST-PUT and LIST-GET, which contain pointers to the last element inserted in the list and the next to be removed, respectively. If the list is empty, both pointers are NULL (0).

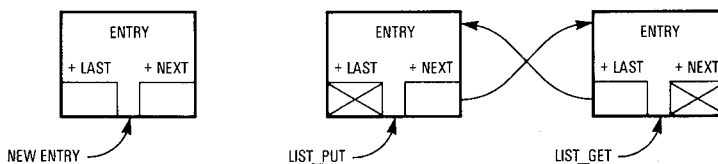
3

The code fragment shown in Figure 3-4 illustrates the insertion of an element in a doubly linked list. The first two instructions load the effective addresses of LIST-PUT and LIST-GET into registers A0 and A1, respectively. The next instruction moves the address of the new element into register D2. Another MOVE instruction moves the address in LIST-PUT into register D0. At label DILOOP, a TST instruction tests the value in D0, and the BEQ instruction branches to the MOVE instruction when D0 is equal to zero. Assuming the list is empty, this MOVE instruction is executed next; it moves the zero in D0 into the NEXT and LAST pointers of the new element. Then the CAS2 instruction moves the address of the new element into both LIST-PUT and LIST-GET, assuming that both of these pointers still contain zero. If not, the BNE instruction branches to the TST instruction at label DILOOP to try again. This time, the BEQ instruction does not branch, and the following MOVE instruction moves the address in D0 to the NEXT pointer of the new element. The CLR instruction clears register D1 to zero, and the MOVE instruction moves the zero into the LAST pointer of the new element. The LEA instruction loads the address of the LAST pointer of the most recently inserted element into register A1. Assuming the LIST-PUT pointer and the pointer in A1 have not been changed, the CAS2 instruction stores the address of the new element into these pointers.

The code fragment to delete an element from a doubly linked list is similar (see Figure 3-5). The first two instructions load the effective addresses of pointers LIST-PUT and LIST-GET into registers A0 and A1, respectively. The MOVE instruction at label DDLOOP moves the LIST-GET pointer into register D1. The BEQ instruction that follows branches out of the routine when the pointer is zero. The MOVE instruction moves the LAST pointer of the element to be deleted into register D2. Assuming this is not the last element in the list, the Z condition code is not set, and the branch to label DDEEMPTY does not occur. The LEA instruction loads the address of the NEXT pointer of the element at the address in D2 into register A2. The next instruction, a CLR instruction, clears register D0 to zero. The CAS2 instruction compares the address in D1 to the LIST-GET pointer and to the address in register A2. If the pointers have not been updated, the CAS2 instruction loads the address in D2 into the LIST-GET pointer and zero into the address in register A2.

DINSERT	LEA LIST_PUT,A0	-ALLOCATE NEW LIST ENTRY, LOAD ADDRESS INTO A2)
	LEA LIST_GET,A1	LOAD ADDRESS OF HEAD POINTER INTO A0
	MOVE.L A2,D2	LOAD ADDRESS OF TAIL POINTER INTO A1
	MOVE.L (A0),D0	LOAD NEW ENTRY POINTER INTO D2
		LOAD POINTER TO HEAD ENTRY INTO D0
DILLOOP	TST.L D0	IS HEAD POINTER NULL (0 ENTRIES IN LIST)?
	BEQ DEMPTY	IF SO, WE NEED ONLY TO ESTABLISH POINTERS
	MOVE.L D0,(NEXT,A2)	PUT HEAD POINTER INTO FORWARD POINTER OF NEW ENTRY
	CLR.L D1	PUT NULL POINTER VALUE IN D1
	MOVE.L D1,(LAST,A2)	PUT NULL POINTER IN BACKWARD POINTER OF NEW ENTRY
	LEA (LAST,D0),A1	LOAD BACKWARD POINTER OF OLD HEAD ENTRY INTO A1
	CAS2.L D0:D1,D2:D2,(A0):(A1)	IF WE STILL POINT TO OLD HEAD ENTRY, UPDATE POINTERS
	BNE DILLOOP	IF NOT, TRY AGAIN
	BRA DIDONE	
DEEMPTY	MOVE.L D0,(NEXT,A2)	PUT NULL POINTER IN FORWARD POINTER OF NEW ENTRY
	MOVE.L D0,(LAST,A2)	PUT NULL POINTER IN BACKWARD POINTER OF NEW ENTRY
	CAS2.L D0:D0,D2:D2,(A0):(A1)	IF WE STILL HAVE NO ENTRIES, SET BOTH POINTERS TO THIS ENTRY
	BNE DILLOOP	IF NOT, TRY AGAIN
DIDONE		SUCCESSFUL LIST ENTRY INSERTION

BEFORE INSERTING NEW ENTRY:



AFTER INSERTING NEW ENTRY:

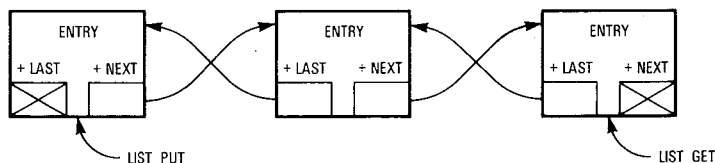
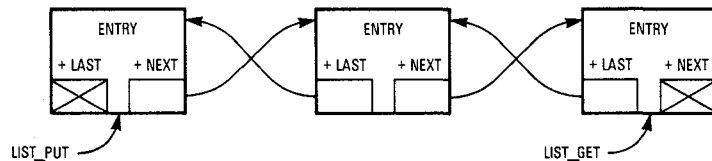


Figure 3-4. Doubly Linked List Insertion

When the list contains only one element, the routine branches to the CAS2 instruction at label DDEEMPTY after moving a zero pointer value into D2. This instruction checks the addresses in LIST_PUT and LIST_GET to verify that no other routine has inserted another element or deleted the last element. Then the instruction moves zero into both pointers, and the list is empty.

DDELETE	LEA LIST_PUT,A0	GET ADDRESS OF HEAD POINTER IN A0
	LEA LIST_GET,A1	GET ADDRESS OF TAIL POINTER IN A1
DDLOOP	MOVE.L (A1),D1	MOVE TAIL POINTER INTO D1
	BEQ DDDONE	IF NO LIST, QUIT
	MOVE.L (LAST,D1),D2	PUT BACKWARD POINTER IN D2
	BEQ DDEEMPTY	IF ONLY ONE ELEMENT, UPDATE POINTERS
	LEA (NEXT,D2),A2	PUT ADDRESS OF FORWARD POINTER IN A2
	CLR.L D0	PUT NULL POINTER VALUE IN D0
	CAS2.L D1:D1,D2:D0,(A1):(A2)	IF BOTH POINTERS STILL POINT TO THIS ENTRY, UPDATE THEM
	BNE DDLOOP	IF NOT, TRY AGAIN
DDEEMPTY	BRA DDDONE	
	CAS2.L D1:D1,D2:D2,(A1):(A0)	IF STILL FIRST ENTRY, SET HEAD AND TAIL POINTERS TO NULL
	BNE DDLOOP	IF NOT, TRY AGAIN
DDDONE		SUCCESSFUL ENTRY DELETION, ADDRESS OF DELETED ENTRY IN D1 (MAY BE NULL)

BEFORE DELETING ENTRY:



AFTER DELETING ENTRY:

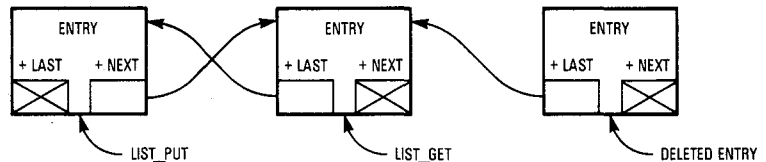


Figure 3-5. Doubly Linked List Deletion

3.5.2 Nested Subroutine Calls

The LINK instruction pushes an address onto the stack, saves the stack address at which the address is stored, and reserves an area of the stack. Using this instruction in a series of subroutine calls results in a linked list of stack frames.

The UNLK instruction removes a stack frame from the end of the list by loading an address into the stack pointer and pulling the value at that address from the stack. When the operand of the instruction is the address of the link address at the bottom of a stack frame, the effect is to remove the stack frame from the stack and from the linked list.

3.5.3 Bit Field Operations

One data type provided by the MC68030 is the bit field, consisting of as many as 32 consecutive bits. A bit field is defined by an offset from an effective address and a width value. The offset is a value in the range of -2^{31} through $2^{31} - 1$ from the most significant bit (bit 7) at the effective address. The width is a positive number, 1–32. The most significant bit of a bit field is bit 0; the bits number in a direction opposite to the bits of an integer.

The instruction set includes eight instructions that have bit field operands. The insert bit field (BFINS) instruction inserts a bit field stored in a register into a bit field. The extract bit field signed (BFEXTS) instruction loads a bit field into the least significant bits of a register and extends the sign to the left, filling the register. The extract bit field unsigned (BFEXTU) also loads a bit field, but zero fills the unused portion of the destination register.

The set bit field (BFSET) instruction sets all the bits of a field to ones. The clear bit field (BFCLR) instruction clears a field. The change bit field (BFCHG) instruction complements all the bits in a bit field. These three instructions all test the previous value of the bit field, setting the condition codes accordingly. The test bit field (BFTST) instruction tests the value in the field, setting the condition codes appropriately without altering the bit field. The find first one in bit field (BFFFO) instruction scans a bit field from bit 0 to the right until it finds a bit set to one and loads the bit offset of the first set bit into the specified data register. If no bits in the field are set, the field offset and the field width is loaded into the register.

An important application of bit field instructions is the manipulation of the exponent field in a floating-point number. In the IEEE standard format, the most significant bit is the sign bit of the mantissa. The exponent value begins at the next most significant bit position; the exponent field does not begin on a byte boundary. The extract bit field (BFEXTU) instruction and the BFTST instruction are the most useful for this application, but other bit field instructions can also be used.

Programming of input and output operations to peripherals requires testing, setting, and inserting of bit fields in the control registers of the peripherals, which is another application for bit field instructions. However, control register locations are not memory locations; therefore, it is not always possible to insert or extract bit fields of a register without affecting other fields within the register.



Another widely used application for bit field instructions is bit-mapped graphics. Because byte boundaries are ignored in these areas of memory, the field definitions used with bit field instructions are very helpful.

3.5.4 Pipeline Synchronization with the NOP Instruction

3

Although the no operation (NOP) instruction performs no visible operation, it serves an important purpose. It forces synchronization of the integer unit pipeline by waiting for all pending bus cycles to complete. All previous integer instructions and floating-point external operand accesses complete execution before the NOP begins. The NOP instruction does not synchronize the FPU pipeline; floating-point instructions with floating-point register operand destinations can be executing when the NOP begins.